

HERMES

Diagnostic & Recommandations



Analyse concurrentielle, diagnostic technique
et feuille de route pour la plateforme d'agents
IA d'acquisition B2B

6 lacunes critiques identifiées

5 recommandations stratégiques priorisées

4 phases de progression sur 12 mois

8

AGENTS IA

9

CONCURRENTS

5

RECOMMANDATIONS

Table des matieres

1. Resume executif	2
2. Etat des lieux du codebase	2
2.1 Architecture generale	2
2.2 Agents IA : etat actuel	3
2.3 Integration LinkedIn	4
3. Analyse concurrentielle	4
3.1 Paysage concurrentiel 2026	4
3.2 Positionnement d'HERMES	5
4. Diagnostic technique detaille	6
4.1 Absence d'orchestration inter-agents	6
4.2 Absence de boucle de feedback	6
4.3 Stockage volatile (localStorage)	6
4.4 Conformite LinkedIn insuffisante	7
4.5 Silo contenu-prospection	7
5. Recommandations strategiques	7
5.1 Implementer un orchestrateur d'agents (PRIORITE CRITIQUE)	7
5.2 Creer une boucle de feedback continue (PRIORITE HAUTE)	8
5.3 Migrer vers une persistance robuste (PRIORITE HAUTE)	8
5.4 Ajouter la conformite LinkedIn proactive (PRIORITE HAUTE)	8
5.5 Etendre au multi-canal (PRIORITE MOYENNE)	9
6. Feuille de route technique	9
6.1 KPIs de suivi	9
7. Synthese et priorisation	10

1. Resume executif

HERMES est un dashboard d'acquisition B2B sur LinkedIn reposant sur 8 agents IA autonomes (Contenu, Qualification, Prospection, Engagement, Veille, Nurturing, Analyse, Reseau). L'application est construite en Next.js 16 avec un store Zustand, une integration OAuth 2.0 LinkedIn et un systeme de simulation d'agents. Malgre une ambition produit considerable et une interface soignee, l'application presente des lacunes significatives par rapport aux concurrents du marche de l'automation LinkedIn en 2026.

Notre analyse identifie 6 lacunes critiques : l'absence d'orchestration inter-agents, l'absence de boucle de feedback, la dependance au contenu generique de l'IA, le silo entre contenu et prospection, l'absence de conformite proactive et le stockage de donnees volatile en localStorage. Ces lacunes placent HERMES en retrait par rapport a des concurrents comme Expandi (\$99/mois), Salesflow (\$99/mois), Dripify (\$10/mois) et HeyReach (\$59/mois), qui disposent de fonctionnalites d'automatisation matures, de sequences multi-canal et d'un CRM integre.

Le rapport propose 5 recommandations strategiques priorisees, un plan d'evolution technique en 4 phases et une feuille de route sur 12 mois pour combler ces ecarts et positionner HERMES comme la reference en matiere d'agents IA d'acquisition B2B.

Constat principal : HERMES a une vision produit ambitieuse et differenciante (agents IA), mais son execution technique reste au stade de prototype fonctionnel. Les concurrents directs proposent des fonctionnalites plus completes et eprouvees a des prix accessibles.

2. Etat des lieux du codebase

2.1 Architecture generale

Le codebase est structure autour d'une application Next.js 16 avec React 19, utilisant le App Router. L'etat global est gere par Zustand avec persistance localStorage (middleware persist). L'interface est construite avec shadcn/ui et Tailwind CSS 4. Le backend utilise des API Routes Next.js avec un systeme d'authentification OAuth 2.0 LinkedIn via cookies httpOnly. La base de donnees est SQLite via Prisma, bien que seuls les modeles User et Post y soient definis, sans reel usage operationnel.

Le coeur du systeme repose sur le fichier appStore.ts (plus de 1200 lignes) qui centralise l'etat des 8 agents, les leads, l'ICP, la configuration des providers IA (10 providers supportes dont Groq, OpenRouter, Anthropic, OpenAI), les metriques et les journaux d'activite. Le fichier agent-runner.ts (plus de 1100 lignes) implemente la logique d'execution de chaque agent avec

un mecanisme de fallback vers la simulation quand aucune cle API n'est configuree.

Composant	Technologie	Role
Frontend	Next.js 16, React 19, Tailwind 4	Interface utilisateur et routage
State Management	Zustand + persist	Etat global avec persistance localStorage
UI Components	shadcn/ui, Radix UI, Framer Motion	Composants accessibles et animes
Backend API	Next.js API Routes	Endpoints REST et OAuth 2.0
AI Gateway	10 providers (Groq, OpenRouter, etc.)	Routage multi-provider unifie
Database	SQLite via Prisma	Persiste User/Post (usage minimal)
LinkedIn Integration	OAuth 2.0 + LinkedIn API	Publication, like, commentaire, feed

Tableau 1 : Stack technique HERMES

2.2 Agents IA : etat actuel

Chaque agent dispose d'un prompt systeme dedie dans agent-runner.ts avec une logique d'execution specifique. Les agents Contenu et Qualification utilisent le LLM pour generer du contenu et scorer les leads, tandis que les agents Engagement, Veille, Nurturing, Analyse et Reseau generent principalement du contenu simule en fallback. Le systeme de simulation (useAgentSimulation.ts) execute les agents en round-robin avec une frequence configurable (x1, x2, x4), mais ne dispose d'aucun veritable orchestrateur coordonnant les dependances entre agents.

Agent	Fonction	Execution reelle	Fallback
01 - Contenu	Generation de posts LinkedIn	LLM (chatCompletion)	Post simule
02 - Qualification	Scoring ICP des leads	LLM + heuristique	Score heuristique
03 - Prospection	Messages personnalises	LLM	DM simule
04 - Engagement	Commentaires sur posts ICP	LLM	Commentaire simule
05 - Veille	Briefings marche	LLM (JSON)	Briefing fige
06 - Nurturing	Suivi leads en attente	LLM	Action simulee

07 - Analyse	Recommandations perf.	LLM (JSON)	Insights fixes
08 - Reseau	Invitations strategiques	LLM	Note simulee

Tableau 2 : Capacites des agents IA

2.3 Integration LinkedIn

L'integration LinkedIn est l'un des points forts du codebase avec un flux OAuth 2.0 complet (auth, callback, disconnect), la gestion des tokens via cookies httpOnly securises, la publication de posts (personnels et pages entreprise), le like et commentaire sur des posts, la consultation du feed, la planification de posts et l'optimisation IA du profil (scoring, suggestions). Cependant, l'API LinkedIn officielle impose des restrictions majeures : limite de 100 000 appels quotidiens, acces restreint au Member Data Platform (MDP), interdiction de scraper les donnees des membres, et scopes limits (openid, profile, email, w_member_social).

Un point critique est le mode simulation du feed : l'API LinkedIn ne permettant pas d'accéder au feed reel d'un utilisateur, l'application genere un feed simule avec 6 posts types. Cette limitation est inherente a l'API LinkedIn et affecte tous les concurrents de la meme maniere, sauf ceux qui contournent les restrictions via des methodes non-officielles (navigateur headless, cookies de session).

3. Analyse concurrentielle

3.1 Paysage concurrentiel 2026

Le marche de l'automation LinkedIn B2B en 2026 est extremement competitif avec plus de 40 outils references. Les prix s'echelonnent de 8,25 USD/mois (Linked Helper) a 1 499 USD/mois (HeyReach Unlimited). Le segment majoritaire se situe entre 50 et 150 USD/mois, avec une tendance marquee vers l'integration de fonctionnalites IA (AI SDR, personnalisation automatique, scoring predictif). Les principaux concurrents d'HERMES se repartissent en trois categories : les outils d'automation pure, les plateformes multi-canal et les solutions AI-native.

Outil	Prix/mois	Forces	Faiblesses
Expandi	\$79-99	Sequences safe, Smart Inbox, scraping profils	Pas de contenu IA, CRM limite
Salesflow	\$99	Multi-comptes, hub CRM, tableaux leads	UI datee, pas de veille marche
Dripify	\$10-59	Prix agressif, sequences drip, A/B test	Fonctionnalites limitees, pas IA

HeyReach	\$59-1499	Scalable agences, warmup, analytics	Cher, complexe pour solos
Lemlist	\$59-99	Multi-canal (email+LinkedIn), videos	Email-first, LinkedIn secondaire
Waalaxy	\$20-40	Simple, onboarding rapide, pas cher	Fonctionnalites basiques
Botdog	\$30-70	Pipeline visuel, rapport auto	Pas de contenu IA autonome
ConnectSafely	\$49-99	API REST complete, contournement restrictions	Technique, pas grand public

Tableau 3 : Comparaison des concurrents directs

3.2 Positionnement d'HERMES

HERMES se différencie par sa vision 'agents IA autonomes' : plutôt que d'automatiser des tâches unitaires (envoi de messages, séquences de connexion), il orchestre 8 agents spécialisés qui couvrent l'intégralité du funnel d'acquisition B2B. Cette approche est unique sur le marché et correspond à la tendance 'Agentic AI' identifiée par Deloitte et Blue Prism pour 2026. Cependant, la différenciation est aujourd'hui théorique : les agents fonctionnent majoritairement en mode simulation, sans coordination réelle entre eux, et sans boucle de feedback qui permettrait au système d'apprendre de ses résultats.

Dimension	Concurrents	HERMES	Ecart
Orchestration agents	Workflows lineaires (trigger/action)	8 agents independants	Critique
Séquences multi-canal	Email + LinkedIn + tel	LinkedIn uniquement	Majeur
CRM integre	Pipeline, tags, filtres	Table leads basique	Majeur
Contenu IA	Templates + personnalisation	Generation LLM (fallback sim.)	Modere
Compliance	Warmup, limites auto, mimetisme	Regles statiques	Critique
Data persistence	Cloud DB, CRM sync	localStorage (volatile)	Critique
Analytics	Dashboards, A/B, ROI	Metriques de base	Majeur
Pricing	\$10-1499/mois	Gratuit (self-hosted)	Avantage
AI Agents	AI SDR (\$500+)	8 agents specialises	Avantage

Tableau 4 : Matrice des écarts HERMES vs Concurrents

4. Diagnostic technique detaille

4.1 Absence d'orchestration inter-agents

Le probleme le plus structurel du codebase est l'absence d'orchestrateur central. Actuellement, `useAgentSimulation.ts` execute les agents en round-robin sans aucune dependance fonctionnelle : l'agent Contenu publie un post sans notifier l'agent Qualification, l'agent Qualification collecte des leads sans alerter l'agent Prospection, et l'agent Analyse genere des recommandations que personne ne consomme. Chaque agent est un ilot autonome qui ne communique pas avec les autres, ce qui annule le benefice theorique d'un systeme multi-agents.

Les fichiers de configuration heartbeat (definis dans `appStore.ts` comme des chaines YAML) decrivent des triggers theoriques (`on_post_published`, `on_linkedin_reply`, `on_no_reply:3d`) mais ceux-ci ne sont jamais parses ni executes. Ils servent uniquement de documentation pour l'utilisateur, sans impact fonctionnel. Pour que le systeme devienne reellement 'agentique', il faudrait implementer un bus d'evenements permettant aux agents de s'envoyer des messages asynchrones et de reagir aux evenements du systeme.

4.2 Absence de boucle de feedback

Aucun agent ne dispose d'un mecanisme d'apprentissage base sur ses resultats. L'agent Contenu genere des posts sans jamais savoir combien d'impressions ou d'engagement ils ont genere. L'agent Prospection envoie des messages sans connaitre le taux de reponse reel. L'agent Analyse produit des recommandations mais ne mesure jamais si leur application ameliore les performances. En consequence, le systeme repete les memes patterns sans jamais s'ameliorer, ce qui est paradoxal pour un produit positionne sur l'IA.

La comparaison avec les concurrents est edifiante : des outils comme Expandi et Lemlist proposent des tests A/B natifs sur les sequences de prospection, avec des algorithmes d'optimisation automatique qui selectionnent les variantes les plus performantes. Dripify offre un tableau de bord analytique detaille permettant d'identifier les messages et les sequences qui convertissent le mieux. HERMES ne dispose d'aucun de ces mecanismes, ce qui rend l'optimisation continue impossible.

4.3 Stockage volatile (localStorage)

L'utilisation de Zustand persist avec `localStorage` comme unique mecanisme de persistance est un risque majeur. Toutes les donnees (leads, configuration ICP, cles API, journaux d'activite, posts generes, metriques) sont stockees dans le navigateur de l'utilisateur. Cela signifie qu'un simple effacement du cache navigateur supprime l'integralite des donnees, qu'il n'y a aucune possibilite de collaboration multi-utilisateurs, et que les cles API des providers IA sont accessibles en clair dans le `localStorage` (vulnerabilite de securite).

La base de données SQLite via Prisma existe dans le codebase (schema.prisma définit les modèles User et Post) mais n'est pas utilisée pour les données opérationnelles. Les leads, les configurations et les métriques ne sont jamais écrits en base, ce qui rend la migration vers un backend robuste d'autant plus complexe qu'il faudra restructurer la logique de persistance de manière significative.

4.4 Conformité LinkedIn insuffisante

L'API LinkedIn impose des restrictions strictes : limite de 100 000 appels quotidiens, interdiction d'accéder aux données des membres sans MDP, restrictions sur le scraping, et limites d'invitation (100/semaine, 50/jour). HERMES ne dispose d'aucun mécanisme de conformité proactive : pas de compteur d'appels API, pas de warmup progressif des comptes, pas de limitation automatique du volume d'actions, pas de détection de risque de ban. Les concurrents comme Salesflow et Expandi intègrent ces protections nativement, avec des systèmes de mimétisme humain (délais aléatoires, variation des horaires, rotation des actions).

4.5 Silo contenu-prospection

L'agent Contenu génère des posts LinkedIn sans aucune boucle de retour vers les agents Qualification et Prospection. En théorie, un post qui génère de l'engagement devrait automatiquement déclencher la collecte des interactions et la qualification des profils qui ont interagi. En pratique, ces agents fonctionnent de manière totalement indépendante. Le contenu publié ne nourrit pas la prospection, et les leads qualifiés n'influencent pas la stratégie éditoriale. Ce silo est d'autant plus dommageable qu'il constitue le cœur de la promesse produite d'HERMES.

5. Recommandations stratégiques

5.1 Implémenter un orchestrateur d'agents (PRIORITE CRITIQUE)

L'orchestrateur est le composant manquant le plus critique. Il doit implémenter un bus d'événements (EventEmitter ou Redis Pub/Sub) permettant aux agents de communiquer de manière asynchrone. Les triggers définis dans les fichiers heartbeat (on_post_published, on_linkedin_reply, on_no_reply:3d) doivent être parsés et exécutés réellement. L'orchestrateur doit gérer les dépendances entre agents (l'agent Qualification ne peut pas scorer des leads avant que l'agent Contenu n'ait publié), les priorités d'exécution et la gestion des conflits.

Architecture proposee : Implementer un AgentOrchestrator base sur un event bus (EventEmitter cote client, WebSocket/Redis cote serveur). Chaque agent emet des evenements (agent.contenu.post_published, agent.qualif.leads_collected) et s'abonne aux evenements des autres agents. Les heartbeats YAML sont parses en regles executables.

5.2 Creer une boucle de feedback continue (PRIORITE HAUTE)

Chaque agent doit mesurer l'impact de ses actions et ajuster son comportement en consequence. L'agent Contenu doit connaitre les impressions, le taux d'engagement et les commentaires de ses posts pour privilegier les formats et sujets qui performant le mieux. L'agent Prospection doit mesurer le taux de reponse par type de message et par secteur pour optimiser ses templates. L'agent Analyse doit non seulement produire des recommandations mais aussi verifier si les recommandations precedentes ont ete appliquees et si elles ont ameliore les resultats.

Concretement, cela necessite la mise en place d'une table Metrics en base de donnees (pas localStorage) avec un historique des actions et de leurs resultats, un systeme de tags A/B pour les posts et messages (permettant de comparer les variantes), et un algorithme d'optimisation qui ajuste les parametres des agents en fonction des performances observees (par exemple, ajuster la temperature du LLM, modifier la longueur des messages, changer les horaires de publication).

5.3 Migrer vers une persistance robuste (PRIORITE HAUTE)

La migration de localStorage vers une base de donnees relationnelle (PostgreSQL via Prisma) est indispensable pour la fiabilite, la securite et la scalabilite du produit. Les cles API doivent etre chiffrees et stockees cote serveur, jamais exposees au client. Les leads, metriques et journaux d'activite doivent etre persistes en base pour permettre l'analyse historique et la collaboration multi-utilisateurs. Cette migration doit etre progressive : commencer par les donnees critiques (leads, metriques) puis etendre aux configurations et journaux.

5.4 Ajouter la conformite LinkedIn proactive (PRIORITE HAUTE)

HERMES doit integrer des mecanismes de protection conformement aux limites LinkedIn : compteur d'appels API avec alertes proches des seuils (80%, 90%, 95% du quota quotidien), warmup progressif des nouveaux comptes (augmentation graduelle du volume d'actions sur 14 jours), mimetisme humain (delais aleatoires entre les actions, variation des horaires de publication et d'envoi, rotation des templates pour eviter la detection de patterns), et detection de risque de ban (surveillance des signaux d'avertissement LinkedIn, mise en pause automatique en cas de suspicion).

5.5 Etendre au multi-canal (PRIORITE MOYENNE)

Pour rivaliser avec Lemlist et HeyReach, HERMES doit etendre ses canaux au-dela de LinkedIn : email (via SendGrid ou Resend), telephone (via Aircall ou RingCentral), et Twitter/X. L'architecture agent permet theoriquement cette extension : chaque canal serait un nouvel agent specialise (Agent Email, Agent Telephonie) qui s'integre a l'orchestrateur. La priorite est l'email, car c'est le canal le plus complementaire de LinkedIn dans une strategie B2B. L'implementation peut s'appuyer sur les memes patterns que l'agent LinkedIn (generation de contenu IA, sequences automatisees, suivi des reponses).

6. Feuille de route technique

Phase	Periode	Objectifs	Livrables
Phase 1 : Fondations	Mois 1-3	Orchestrateur, DB migration, conformite	AgentOrchestrator, PostgreSQL, API tracker
Phase 2 : Feedback	Mois 4-6	Boucle feedback, A/B testing, analytics	Metrics DB, A/B engine, dashboard ROI
Phase 3 : Multi-canal	Mois 7-9	Email, CRM integre, tableaux leads avance	Agent Email, CRM views, pipeline avance
Phase 4 : Scale	Mois 10-12	Multi-utilisateurs, agences, API publique	Auth multi-user, workspace agences, API v1

Tableau 5 : Feuille de route sur 12 mois

La Phase 1 est la plus critique car elle pose les fondations techniques sans lesquelles les phases suivantes sont impossibles. L'orchestrateur doit etre implemente en premier, car il est le prealable a la boucle de feedback et a la coordination multi-canal. La migration PostgreSQL est egalement prioritaire car elle conditionne la persistance des metriques et la possibilite de multi-utilisateurs.

6.1 KPIs de suivi

KPI	Cible Phase 1	Cible Phase 4
Taux d'execution reelle (vs simulation)	40%	85%
Temps de reaction inter-agents	< 5 min	< 30 sec
Couverture API LinkedIn	60%	95%
Taux de conformite LinkedIn	95%	99.5%

Retention donnees (vs localStorage)	100% (PostgreSQL)	100% + backup
Canaux actifs	1 (LinkedIn)	3+ (LinkedIn, email, tel)

Tableau 6 : KPIs de progression

7. Synthèse et priorisation

HERMES dispose d'un positionnement produit differenciant sur un marche en pleine mutation vers l'Agentic AI. La vision des 8 agents specialises couvrant l'integralite du funnel d'acquisition B2B est unique et correspond aux tendances identifiees par Deloitte, Blue Prism et les analyses de marche pour 2026. Cependant, l'execution actuelle reste au stade de prototype fonctionnel avec des lacunes techniques qui limitent serieusement la valeur delivree aux utilisateurs.

Les 5 recommandations priorisees dans ce rapport visent a combler methodiquement ces ecarts : l'orchestrateur d'agents (critique), la boucle de feedback (haute), la persistance robuste (haute), la conformite LinkedIn proactive (haute) et l'extension multi-canal (moyenne). La feuille de route sur 12 mois propose une progression en 4 phases qui respecte les dependances techniques et minimise les risques. En suivant cette trajectoire, HERMES peut evoluer d'un prototype ambitieux vers une plateforme de reference en matiere d'agents IA d'acquisition B2B.

Recommandation	Priorite	Impact	Effort	Phase
Orchestrateur d'agents	Critique	Tres haut	2-3 semaines	1
Boucle de feedback	Haute	Haut	2 semaines	2
Persistance PostgreSQL	Haute	Haut	1-2 semaines	1
Conformite LinkedIn	Haute	Haut	1 semaine	1
Extension multi-canal	Moyenne	Moyen	3-4 semaines	3

Tableau 7 : Matrice de priorisation des recommandations

Message cle : HERMES a un avantage strategique considerable avec sa vision agentique. Mais sans orchestrateur et sans boucle de feedback, les agents ne sont que des scripts independants. L'enjeu n'est pas d'ajouter des fonctionnalites, mais de connecter les agents entre eux pour creer un systeme reellement intelligent.